

Dataset Report: MiniDB Song Collection

Generation Process

This dataset is a generated collection of 1,000,000 song records, created specifically for the "MiniDB" project. The data was generated using a Python script leveraging the faker, random, and uuid libraries.

The primary design goal was to create a dataset that fulfills all project requirements:

- Contains 1,000,000 records.
- Includes 8 attributes covering numeric, textual, categorical, and relational types.
- Features a built-in graph structure (artist collaborations).
- Provides predictable data patterns for testing analytics, range queries, and indexing.

Dataset Schema

The dataset is provided as a songs_dataset.csv file with the following 8-attribute schema:

Attribute	Data Type	Field Type	Description & Project Purpose
song_id	String (UUID)	Categorical	Primary Key. A guaranteed unique identifier (UUIDv4) for each song. Essential for efficient search, modification, and deletion operations.
song_title	String	Textual	The title of the song. A simple textual field.
artist_name	String	Categorical	Graph Node. The primary artist. This field is drawn from a fixed pool of

			7,000 unique names to ensure relationships are created. Each unique artist forms a node in the collaboration graph.
featured_artists	String (List)	Relational	Graph Edges
album_title	String	Textual	The title of the song's album.
release_year	Integer	Numeric	Range Query Target. A numeric field with a uniform distribution of integers between 1970 and 2025. This is ideal for testing the performance and correctness of range queries (e.g., "find all songs from 1980-1990").
duration_ms	Integer	Numeric	Analytics Target. A numeric field with a uniform distribution of integers representing song length in milliseconds (from 2 to 6 minutes). This field is intended for analytical operations like min, max, average, and

			median.
genre	String	Categorical	Analytics Target. A categorical field drawn from a fixed pool of 18 genres (e.g., 'Pop', 'Rock', 'Hip-Hop'). This is ideal for testing analytical queries like top-K (e.g., "find the top 5 most common genres").

Key Data Patterns & Features

The data was generated with specific, intentional patterns to facilitate testing every part of the project:

- **Graph Structure:** The relationship between artist_name (node) and featured_artists (edges) forms a "collaboration network." Because all artists are drawn from a shared pool, this graph will have meaningful connections, clusters, and hubs.
- **Graph Sparsity:** The featured_artists column is intentionally sparse. Approximately **75% of songs have no features** (an empty string), ~20% have one, and ~5% have two. This mimics a real-world pattern where collaborations are special, making graph pathfinding more interesting.
- **Query-Friendly Distributions:** The release_year and duration_ms fields use a **uniform distribution**. This means a query on any 10-year period will return a predictable and roughly equal number of results, making it easy to benchmark the performance of your range queries.
- **Groupable Data:** The genre and artist_name fields are pulled from fixed pools, guaranteeing a high degree of repetition. This is perfect for aggregation, top-K analytics, and testing the efficiency of any secondary indexes.